2 hours

THE UNIVERSITY OF MANCHESTER

Introduction to Programming II

Resit Exam, Summer 2025

Instructions for candidates

Answer ALL questions and upload your responses to Gradescope in accordance with the online instructions.

The examination is divided into TWO Sections and is worth a total of 40 marks:

- Section 1 is worth 25 marks (62.5%)
- Section 2 is worth 15 marks (37.5%)

You should write all of your code in packages, in accordance with the UML given in the exam questions.

Uploading your responses to Gradescope

You should upload a zip file containing your top-level package directory, in this case **`traitors`**. For example, if the exam requires you to write a class named `Game` in the **`traitors`** package, and you've been working on the file:

```
S:\traitors\Game.java
```

Then you should upload a zip of the **`traitors`** directory to Gradescope.

You can upload your solutions as many times as you wish during the exam. Each time you upload a solution your mark/test output will be updated.

Your final mark will be determined by the highest scoring solution that you upload to Gradescope. You **MUST** upload a solution to Gradescope before the exam ends in order to receive a non-zero mark.

Additional permitted materials

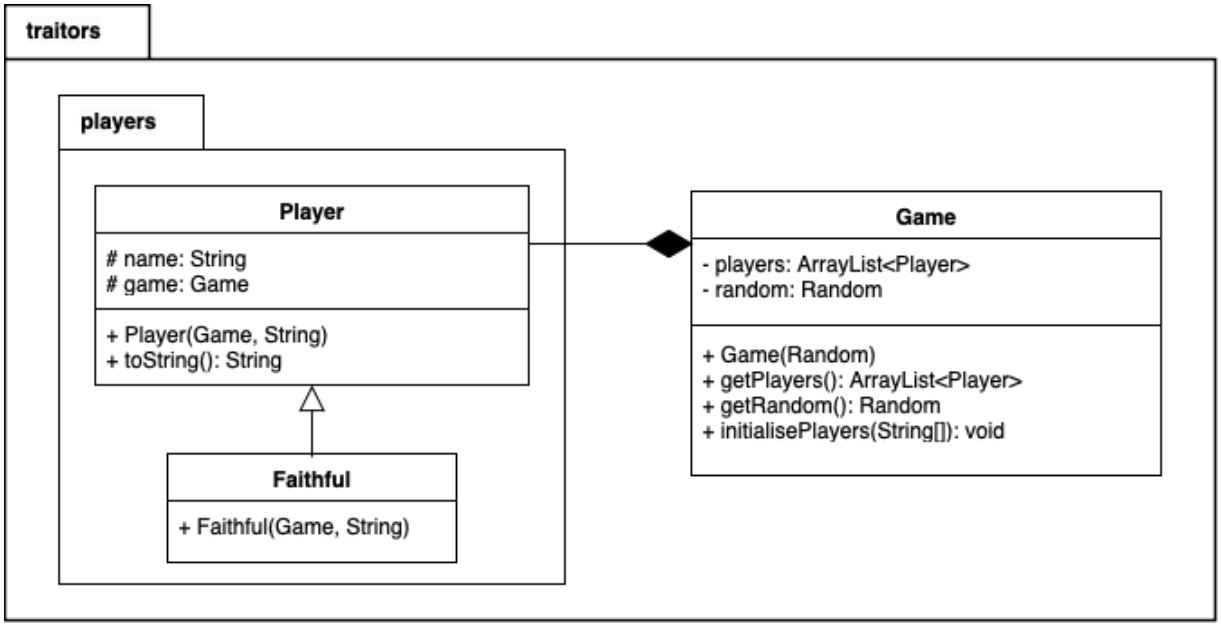
Answer this exam on the computer provided.

The use of calculators is NOT permitted in this exam.

Access to the Java API (and JavaFX APIs where appropriate) will be possible in the exam. Other websites will NOT be available during this exam. You should not attempt to view any other content during this exam.

Section 1. Answer ALL PARTS of BOTH questions in this section.

The following UML diagram represents entities involved in the popular TV game format "The Traitors". In this game, each player joins the game as a `Faithful`.



Question 1

(a) Download the initial code file from Blackboard. This file contains:

- The `Player` class -- a full implementation consistent with the above UML.
- An outline of the `Game` class (i.e., the visibility modifier, class name, opening and closing braces).

(b) For the `Game` class, implement:

i. The `players` and `random` attributes. The `random` attribute of the `Game` class is an instance of `java.util.Random`.

ii. The constructor, which should set the `random` attribute to the instance passed to the constructor. It should initialise the `players` attribute as an empty `ArrayList` of `Player` objects.

iii. The `getRandom` method, a getter for the `random` attribute.

iv. The `getPlayers` method, a getter for the `players` attribute.

[3 marks]

(c) Implement the `Faithful` subclass of the `Player` class.

[2 marks]

(d) Implement the `initialisePlayers` method of the `Game` class.

This method takes as a parameter an array of `String`s, each one is the `name` of a `Player` joining the game. The method should create one new instance of the `Faithful` class for each `name` (using the `Game` instance itself as the `game` parameter) and add them to the `players` attribute.

The order of players added should be consistent with the ordering of names in the `String` array that was passed as a parameter.

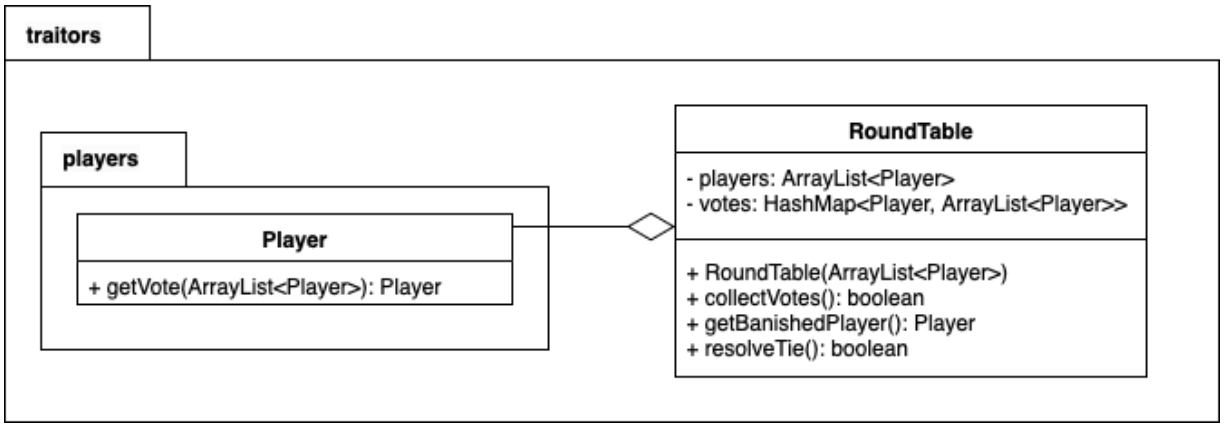
DO NOT ADD A CALL TO `initialisePlayers` TO THE `Game` CLASS CONSTRUCTOR.

[4 marks]

Question 2

Each day in the game, the players have a round table discussion to try and identify the traitors in the group.

Note that both the `RoundTable` class in the below UML, and the `Game` class in the previous UML, contain an attribute named `players`. All subsequent questions will therefore make it clear which attribute is being referred to (e.g., "the `players` list of the `Game` class").



(a) Implement the attributes and constructor of the `RoundTable` class. The constructor takes as a parameter the list of all players still in the game (to be stored in the `players` attribute of this same class). The constructor should initialise, but not populate, the `votes` attribute.

[3 marks]

(b) Implement the `getVote` method of `Player`. This method randomly selects and returns one `Player` from the list of players received in its parameter – the returned value is the `Player` that the current `Player` object is voting for.

Note that:

- The method should call the `nextInt` method of the `Random` instance in the player's `game` attribute to determine an index into the `ArrayList` passed as a parameter. The `Player` at this index is the one that this player will vote for.

- `Player`s should not vote for themselves (if `nextInt` returns the index of the `Player` casting the vote, then further calls to `nextInt` should be used until another value is returned).

[2 marks]

(c) Implement the `collectVotes` method of the `RoundTable` class. This method calls `getVote` on each player in the `players` list of the `RoundTable` class (in order) and stores the result in `votes`. The keys of the `votes` `HashMap` are instances of players who have been voted for, and the values is a list of `Player`s who voted for them (in the order that the votes were cast).

The return value of `collectVotes` should be `true` if a single `Player` has received more votes than any other, or `false` if more than one `Player` is tied for the most votes received.

[4 marks]

(d) Implement the `resolveTie` method of the `RoundTable` class. This method calls `getVote` on each player in the `players` list of the `RoundTable` class (in order) and stores the result in `votes` (as per `collectVotes`).

Note that:

- The method should first identify the players who are tied for having the most votes following a previous call to `collectVotes` or `resolveTie` (based on the contents of the `votes` attribute).
- It should then clear the values of the `votes` attribute **ONLY** for keys of players who were tied for having the most votes.
- This method should pass the `getVote` method a parameter containing those players who were tied with the most votes. These players should be ordered in accordance with their order in the `players` list of the `RoundTable` class (i.e. if `players` contains `[a, b, c]`, and `a` and `c` are tied for the most votes, then the parameter passed to `getVote` should be ordered `[a, c]`).

- The method calls `getVote` on all players in the **`RoundTable`**, including those tied with the most votes.

The return value of `resolveTie` should be `true` if a single player from those who were previously tied has now received more votes than any other, or `false` if there is still more than one player tied for the most votes received. **DO NOT IMPLEMENT `resolveTie` RECURSIVELY.**

[5 marks]

(e) Implement the `getBanishedPlayer` method of the **`RoundTable`** class. This method should return the single **`Player`** with the most votes. If this cannot be determined (e.g. if no votes have been collected, or if there is a tie), then the method should result in a `java.lang.IllegalStateException`.

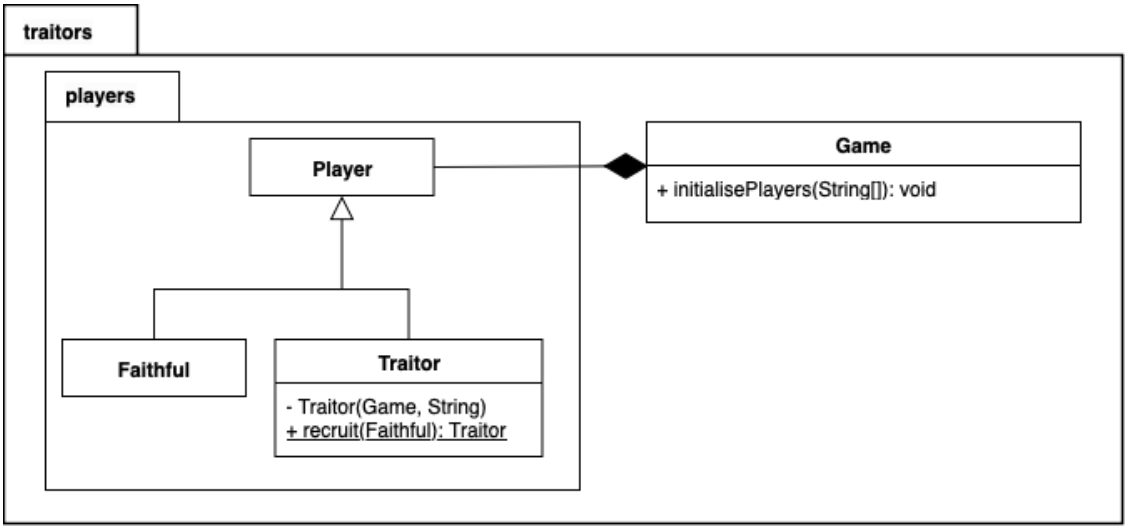
[2 marks]

THIS PAGE IS INTENTIONALLY LEFT BLANK

SECTION 2 BEGINS ON PAGE 8

Section 2. Answer ALL PARTS of BOTH questions in this section.

Early in the game, some `Player`s are secretly selected to be `Traitor`s.



Question 1

(a) Implement the `Traitor` subclass of the `Player` class.

Note that the usual way to make a `Player` into a `Traitor` is to call the `recruit` method – this takes an existing `Faithful` player as its parameter and returns a new `Traitor` object with the same attribute values as the passed `Faithful` player.

[3 marks]

(b) Modify the `recruit` method of the `Traitor` class so that when a Traitor is recruited, the original `Faithful` player is removed from the `players` attribute of the `Game` class, and the new `Traitor` replaces them. The `Traitor` should appear in the same position in the list as the `Faithful` that they have replaced.

[4 marks]

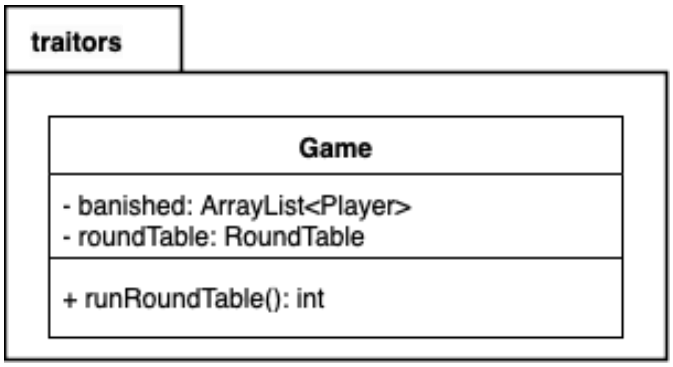
(c) Modify the `initialisePlayers` method of the **`Game`** class, so that after creating all players as **`Faithful`**, **THREE** players are randomly selected and recruited as **`Traitor`**s.

Note that:

- The method should use the `nextInt` method of the `Random` class to identify three different indices into the `players` attribute of the **`Game`** class, these are the **`Faithful`** to be recruited. You may need to call `nextInt` more than three times -- if `nextInt` returns the index of a player that has already been selected for recruitment, then you should continue calling `nextInt` until have recruited three different **`Faithful`**.
- It should then call the `recruit` method of the **`Traitor`** class to convert each identified **`Faithful`** instance to a **`Traitor`**.
- **YOU SHOULD NOT ADD A CALL TO `initialisePlayers` TO THE `Game` CLASS CONSTRUCTOR.**

[2 marks]

Question 2
The following UML shows some additions to the existing `Game` class.



(a) Implement the `runRoundTable` method of the `Game` class. This method should:

- Create a new `RoundTable` with the current list of players, storing the new object in the variable `roundTable`. The ordering of players passed to the `RoundTable` should be consistent with the ordering of the `players` attribute of the `Game` class, and the `RoundTable` class should maintain this order in its own `players` attribute
- Call `collectVotes` once, and then call `resolveTie` zero or more times in order to ensure that a single player has received the most votes.
- Return the number of cycles of voting each player participated in to get this result (i.e., if `collectVotes` resulted in no ties, then the return value is 1; if `resolveTie` was called, then the return value is the number of calls made to `resolveTie` plus one).

[4 marks]

(b) Update the `runRoundTable` method of the `Game` class. Once all ties have been resolved, this method should remove the player with the most votes (identified via `getBanishedPlayer`) from the `players` attribute of the `Game` class, and should add them to `banished`. The order of the `players` attribute should remain otherwise unchanged. The order of the `banished` attribute is unspecified (i.e. you can order this any way you wish).

[2 marks]

END OF EXAMINATION